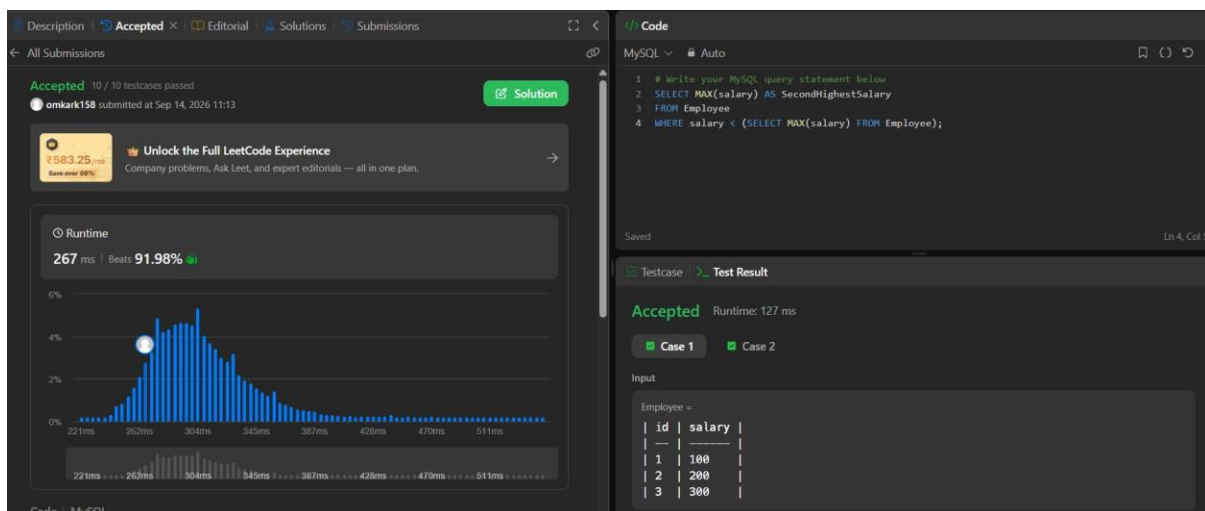


11)



MongoDB -

1. Practice CRUD operations on Mongo. 1. Create two collections -- emp and dept. 2. Insert the emp and dept records. 3. Increase salary of dept 10 emps by 100. 4. Display emp name, job, deptno, sal in sorted order of deptno, job, sal (desc) for all emps whose sal more than 1000. 5. Delete the dept with id 40.

2. Learn grouping operations using Mongo aggregation pipeline 1. Display job wise total sal, max sal, min sal, and avg sal for all jobs other than PRESIDENT. 2. Display emp name, dept name, income (sal + comm) in sorted order of their income.

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91879>cmd
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91879>mongosh
Current Mongosh Log ID: 6aaa8ae6e80df6ab8686b01c
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.2
Using MongoDB: 8.2.5
Using Mongosh: 2.3.2
mongosh 2.10.0 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-09-12T08:24:09.550+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
```

11)

```
test> use dbt_learning
switched to db dbt_learning
dbt_learning> db.dept.insertMany([
...   {
...     deptno: 10,
...     dname: "ACCOUNTING",
...     loc: "NEW YORK"
...   },
...   {
...     deptno: 20,
...     dname: "RESEARCH",
...     loc: "DALLAS"
...   },
...   {
...     deptno: 30,
...     dname: "SALES",
...     loc: "CHICAGO"
...   },
...   {
...     deptno: 40,
...     dname: "OPERATIONS",
...     loc: "BOSTON"
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aaa8bc2e80df6ab8686b01d'),
    '1': ObjectId('6aaa8bc2e80df6ab8686b01e'),
    '2': ObjectId('6aaa8bc2e80df6ab8686b01f'),
    '3': ObjectId('6aaa8bc2e80df6ab8686b020')
  }
}
dbt_learning> db.dept.find()
[
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01d'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'NEW YORK'
  },

```

```

    _id: ObjectId('6aaa8bc2e80df6ab8686b01e'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'DALLAS'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01f'),
    deptno: 30,
    dname: 'SALES',
    loc: 'CHICAGO'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b020'),
    deptno: 40,
    dname: 'OPERATIONS',
    loc: 'BOSTON'
  }
]
dbt_learning> db.emp.insertMany([
...   {
...     empno: 7369,
...     ename: "SMITH",
...     job: "CLERK",
...     deptno: 20,
...     sal: 800,
...     comm: 0
...   },
...   {
...     empno: 7499,
...     ename: "ALLEN",
...     job: "SALESMAN",
...     deptno: 30,
...     sal: 1600,
...     comm: 300
...   },
...   {
...     empno: 7521,
...     ename: "WARD",
...     job: "SALESMAN",
...     deptno: 30,
...     sal: 1250,
...   }
... ])

```

11)

```
...      ename: "KING",
...      job: "PRESIDENT",
...      deptno: 10,
...      sal: 5000,
...      comm: 0
...    },
...    {
...      empno: 7844,
...      ename: "TURNER",
...      job: "SALESMAN",
...      deptno: 30,
...      sal: 1500,
...      comm: 0
...    },
...    {
...      empno: 7876,
...      ename: "ADAMS",
...      job: "CLERK",
...      deptno: 20,
...      sal: 1100,
...      comm: 0
...    },
...    {
...      empno: 7900,
...      ename: "JAMES",
...      job: "CLERK",
...      deptno: 30,
...      sal: 950,
...      comm: 0
...    },
...    {
...      empno: 7902,
...      ename: "FORD",
...      job: "ANALYST",
...      deptno: 20,
...      sal: 3000,
...      comm: 0
...    },
...    {
...      empno: 7934,
...      ename: "MILLER",
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aaa8be5e80df6ab8686b021'),
    '1': ObjectId('6aaa8be5e80df6ab8686b022'),
    '2': ObjectId('6aaa8be5e80df6ab8686b023'),
    '3': ObjectId('6aaa8be5e80df6ab8686b024'),
    '4': ObjectId('6aaa8be5e80df6ab8686b025'),
    '5': ObjectId('6aaa8be5e80df6ab8686b026'),
    '6': ObjectId('6aaa8be5e80df6ab8686b027'),
    '7': ObjectId('6aaa8be5e80df6ab8686b028'),
    '8': ObjectId('6aaa8be5e80df6ab8686b029'),
    '9': ObjectId('6aaa8be5e80df6ab8686b02a'),
    '10': ObjectId('6aaa8be5e80df6ab8686b02b'),
    '11': ObjectId('6aaa8be5e80df6ab8686b02c'),
    '12': ObjectId('6aaa8be5e80df6ab8686b02d'),
    '13': ObjectId('6aaa8be5e80df6ab8686b02e')
  }
}
{ deptno: 10 },
... { $inc: { sal: 100 } }
... )
Uncaught:
SyntaxError: Unexpected token (1:18)

> 1 | { deptno: 10 },
  |   ^
  2 | { $inc: { sal: 100 } }
  3 | )
  4 |

dbt_learning> db.EMP.updateMany(
...   { deptno: 10 },
...   { $inc: { sal: 100 } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

11)

```
dbt_learning> db.emp.find(
...   { deptno: 10 },
...   { _id: 0, ename: 1, deptno: 1, sal: 1 }
... )
[
  { ename: 'CLARK', deptno: 10, sal: 2550 },
  { ename: 'KING', deptno: 10, sal: 5100 },
  { ename: 'MILLER', deptno: 10, sal: 1400 }
]
dbt_learning> db.emp.find(
...   { sal: { $gt: 1000 } },
...   {
...     _id: 0,
...     ename: 1,
...     job: 1,
...     deptno: 1,
...     sal: 1
...   }
... ).sort({
...   deptno: 1,
...   job: 1,
...   sal: -1
... })
[
  { ename: 'MILLER', job: 'CLERK', deptno: 10, sal: 1400 },
  { ename: 'CLARK', job: 'MANAGER', deptno: 10, sal: 2550 },
  { ename: 'KING', job: 'PRESIDENT', deptno: 10, sal: 5100 },
  { ename: 'SCOTT', job: 'ANALYST', deptno: 20, sal: 3000 },
  { ename: 'FORD', job: 'ANALYST', deptno: 20, sal: 3000 },
  { ename: 'ADAMS', job: 'CLERK', deptno: 20, sal: 1100 },
  { ename: 'JONES', job: 'MANAGER', deptno: 20, sal: 2975 },
  { ename: 'BLAKE', job: 'MANAGER', deptno: 30, sal: 2850 },
  { ename: 'ALLEN', job: 'SALESMAN', deptno: 30, sal: 1600 },
  { ename: 'TURNER', job: 'SALESMAN', deptno: 30, sal: 1500 },
  { ename: 'WARD', job: 'SALESMAN', deptno: 30, sal: 1250 },
  { ename: 'MARTIN', job: 'SALESMAN', deptno: 30, sal: 1250 }
]
dbt_learning> db.dept.deleteOne({
...   deptno: 40
... })
```

```
dbt_learning> db.dept.deleteOne({
...   deptno: 40
... })
{ acknowledged: true, deletedCount: 1 }
dbt_learning> db.dept.find()
[
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01d'),
    deptno: 10,
    dname: 'ACCOUNTING',
    loc: 'NEW YORK'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01e'),
    deptno: 20,
    dname: 'RESEARCH',
    loc: 'DALLAS'
  },
  {
    _id: ObjectId('6aaa8bc2e80df6ab8686b01f'),
    deptno: 30,
    dname: 'SALES',
    loc: 'CHICAGO'
  }
]
dbt_learning> db.emp.aggregate([
...   {
...     $match: {
...       job: { $ne: "PRESIDENT" }
...     }
...   },
...   {
...     $group: {
...       _id: "$job",
...       total_sal: { $sum: "$sal" },
...       max_sal: { $max: "$sal" },
...       min_sal: { $min: "$sal" },
...       avg_sal: { $avg: "$sal" }
...     }
...   }
... ])
```

11)

```
[
  {
    _id: 'MANAGER',
    total_sal: 8375,
    max_sal: 2975,
    min_sal: 2550,
    avg_sal: 2791.6666666666665
  },
  {
    _id: 'CLERK',
    total_sal: 4250,
    max_sal: 1400,
    min_sal: 800,
    avg_sal: 1062.5
  },
  {
    _id: 'SALESMAN',
    total_sal: 5600,
    max_sal: 1600,
    min_sal: 1250,
    avg_sal: 1400
  },
  {
    _id: 'ANALYST',
    total_sal: 6000,
    max_sal: 3000,
    min_sal: 3000,
    avg_sal: 3000
  }
]
dbt_learning> db.emp.aggregate([
...   {
...     $lookup: {
...       from: "dept",
...       localField: "deptno",
...       foreignField: "deptno",
...       as: "department"
...     }
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { emp_name: 'SMITH', dept_name: 'RESEARCH', income: 800 },
  { emp_name: 'JAMES', dept_name: 'SALES', income: 950 },
  { emp_name: 'ADAMS', dept_name: 'RESEARCH', income: 1100 },
  { emp_name: 'MILLER', dept_name: 'ACCOUNTING', income: 1400 },
  { emp_name: 'TURNER', dept_name: 'SALES', income: 1500 },
  { emp_name: 'WARD', dept_name: 'SALES', income: 1750 },
  { emp_name: 'ALLEN', dept_name: 'SALES', income: 1900 },
  { emp_name: 'CLARK', dept_name: 'ACCOUNTING', income: 2550 },
  { emp_name: 'MARTIN', dept_name: 'SALES', income: 2650 },
  { emp_name: 'BLAKE', dept_name: 'SALES', income: 2850 },
  { emp_name: 'JONES', dept_name: 'RESEARCH', income: 2975 },
  { emp_name: 'SCOTT', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'FORD', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'KING', dept_name: 'ACCOUNTING', income: 5100 }
]
dbt_learning> |
```

```
...     income: {
...       $add: ["$sal", "$comm"]
...     }
...   },
...   {
...     $sort: {
...       income: 1
...     }
...   }
... ])
[
  { emp_name: 'SMITH', dept_name: 'RESEARCH', income: 800 },
  { emp_name: 'JAMES', dept_name: 'SALES', income: 950 },
  { emp_name: 'ADAMS', dept_name: 'RESEARCH', income: 1100 },
  { emp_name: 'MILLER', dept_name: 'ACCOUNTING', income: 1400 },
  { emp_name: 'TURNER', dept_name: 'SALES', income: 1500 },
  { emp_name: 'WARD', dept_name: 'SALES', income: 1750 },
  { emp_name: 'ALLEN', dept_name: 'SALES', income: 1900 },
  { emp_name: 'CLARK', dept_name: 'ACCOUNTING', income: 2550 },
  { emp_name: 'MARTIN', dept_name: 'SALES', income: 2650 },
  { emp_name: 'BLAKE', dept_name: 'SALES', income: 2850 },
  { emp_name: 'JONES', dept_name: 'RESEARCH', income: 2975 },
  { emp_name: 'SCOTT', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'FORD', dept_name: 'RESEARCH', income: 3000 },
  { emp_name: 'KING', dept_name: 'ACCOUNTING', income: 5100 }
]
dbt_learning> |
```

CDAC Admission Process Case Study

Design database for CDAC Admission System. Create appropriate tables and insert sample records. Write a Stored Procedure for Round1 allocation. Write a Stored Procedure for Round2 allocation. Note: 1. Create ER diagram following Normalization process. 2. Study database constraints in depth: PRIMARY KEY, FOREIGN-KEY, UNIQUE, CHECK, DEFAULT Apply appropriate constraints in CDAC admission system. 3. Learn about composite keys and candidate keys Identify keys in CDAC admission system. 4. Explore AUTO_INCREMENT / SEQUENCES Use appropriate AUTO_INCREMENT keys in CDAC admission system. 5. Study control structures: IF-ELSE, CASE, LOOPS (WHILE, REPEAT, LOOP) Use appropriate control

11)

structures in CDAC admission system stored procedures. 6. Learn stored procedure parameters: IN, OUT, IN-OUT. Use appropriate parameters for CDAC admission system stored procedures. 7. Understand cursors and their usage in following stored procedures. Make appropriate use of cursor in CDAC admission system stored procedures. 8. Explore error handling in stored procedures Implement appropriate error handling in CDAC admission system stored procedures